

St. Francis Institute of Technology, Mumbai-400 103
Department Of Information Technology

A.Y. 2025-2026

Class: BE-ITA/B, Semester: VIII

Subject: BlockChain Lab

Experiment – 3

1. **Aim:** To implement smart contract using Remix IDE and deploy it using MetaMask.
2. **Objective:** To ...
 - explore working of Local ethereum network.
 - explore working of MetaMask wallet
 - test a smart contract by using Remix IDE
3. **Lab outcome:** After performing the experiment, the students will be able to **implement** smart contracts in Ethereum using different development frameworks (PO3, PSO2, BL3)
4. **Prerequisite:**
 - Fundamental knowledge of blockchain
 - Knowledge of the Ethereum platform and Remix IDE
 - Familiarity with the Solidity programming language

5. Requirements: The following are the requirements –
Remix IDE, MetaMask wallet etc.

6. Pre-Experiment Theory:

What is MetaMask?

MetaMask is a browser plugin that serves as a cryptocurrency wallet and enables users to ~~inter~~ **interact** and other ERC-20 tokens. It also enables users to access the Web 3 ecosystem of ~~app~~ **applications** on browser integration and good design to serve as one of the main gateways to the world of Web3, decentralized finance (DeFi) and NFTs.

**7. Laboratory
Exercise Steps to be
implemented.**

To Follow the procedure given below to build smart contract in Remix IDE

1. Download and install the official MetaMask extension/ plugin.
2. Open Remix IDE in Google Chrome.
3. Write smart contract by creating new file under contract folder, with .sol extension.
4. Write your contract code using solidity language.
5. Click on compiler icon, choose compiler version or keep default and click on compile button.
6. For deployment...

In Remix IDE choose environment Injected Provider- MetaMask and then deploy the said contract.

7. Under deployed contract, get the output of your contract.

Program Code

Write a smart contract StudentMarks.sol to create student database with attributes, id, name, dept, sub1marks, sub2Marks etc. and deploy it using Remix and MetaMask.

**Post Experimental Exercise-
Questions:**

- List down the details of different MetaMask networks
- Create any one game using smart contract and deploy it.

A. Results/Observations/Program output:

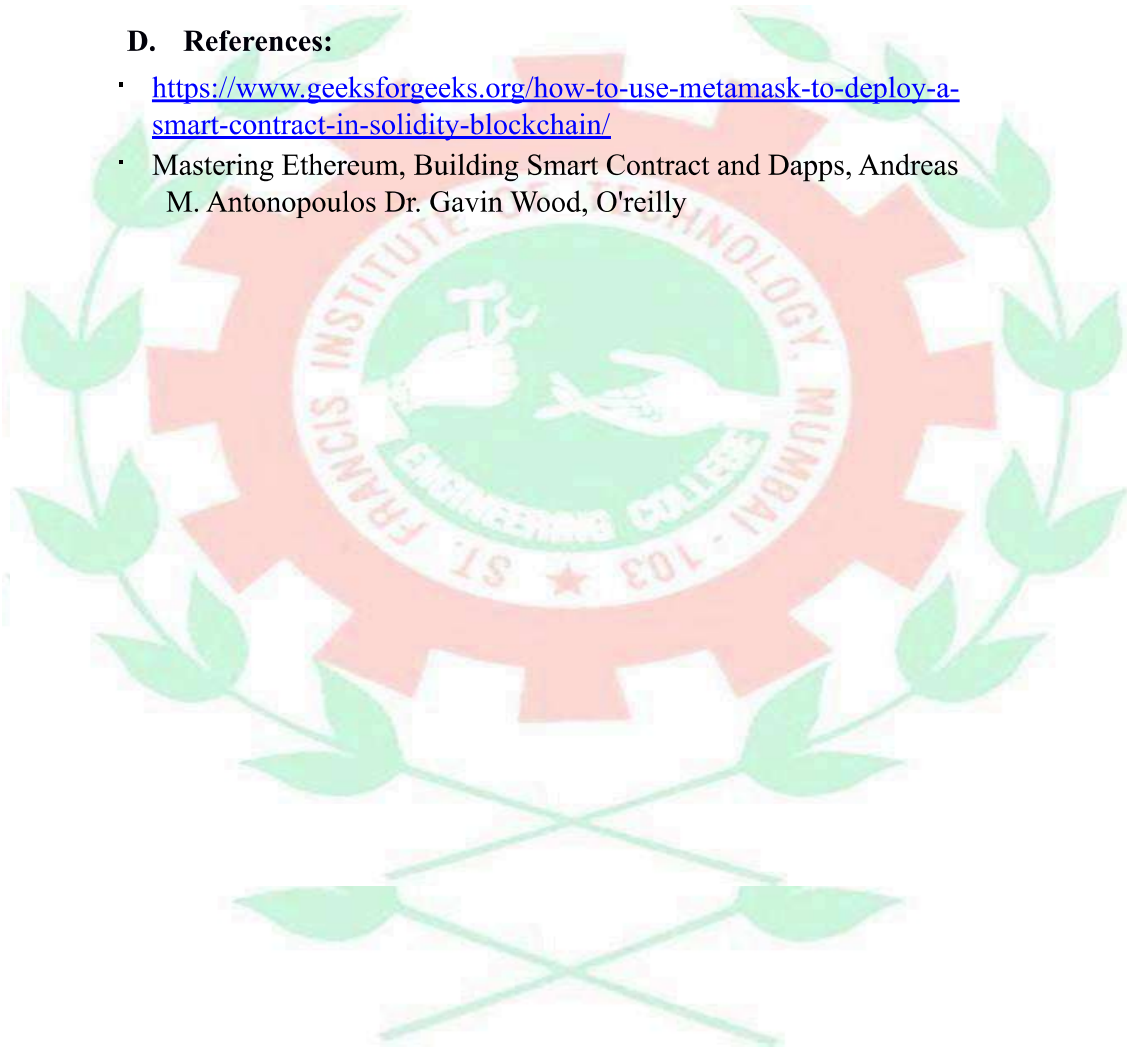
Present the program input/output results if any and comment on the same.

B. Conclusion:

- Write what was performed in the experiment.
- Write which tools you used to perform the experiment
- Write what you inferred from the output obtained.

D. References:

- <https://www.geeksforgeeks.org/how-to-use-metamask-to-deploy-a-smart-contract-in-solidity-blockchain/>
- Mastering Ethereum, Building Smart Contract and Dapps, Andreas M. Antonopoulos Dr. Gavin Wood, O'reilly



Program Code

Write a smart contract StudentMarks.sol to create student database with attributes, id, name, dept, sub1marks, sub2Marks etc. and deploy it using Remix and MetaMask.

Code:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.18;

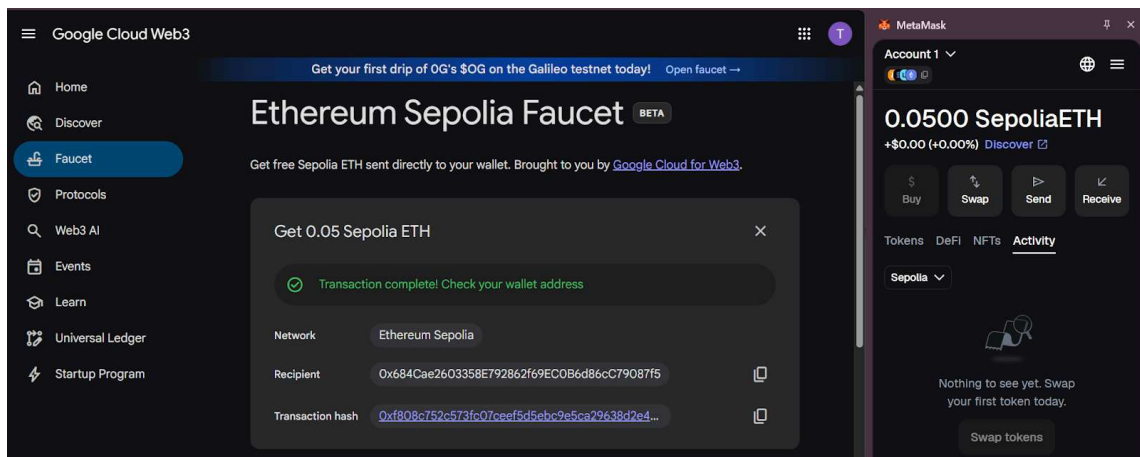
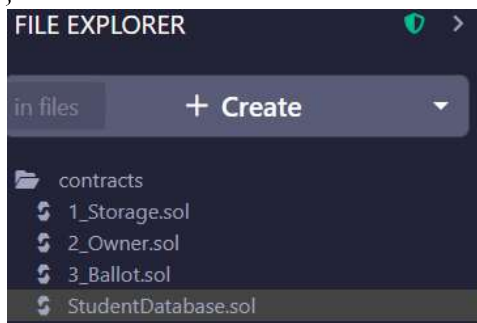
contract StudentMarks {
    struct Student {
        uint id;
        string name;
        string dept;
        uint sub1Marks;
        uint sub2Marks;
    }
    mapping(uint => Student) public students;
    uint public studentCount;

    // Add Student
    function addStudent(
        uint _id,
        string memory _name,
        string memory _dept,
        uint _sub1Marks,
        uint _sub2Marks
    ) public {

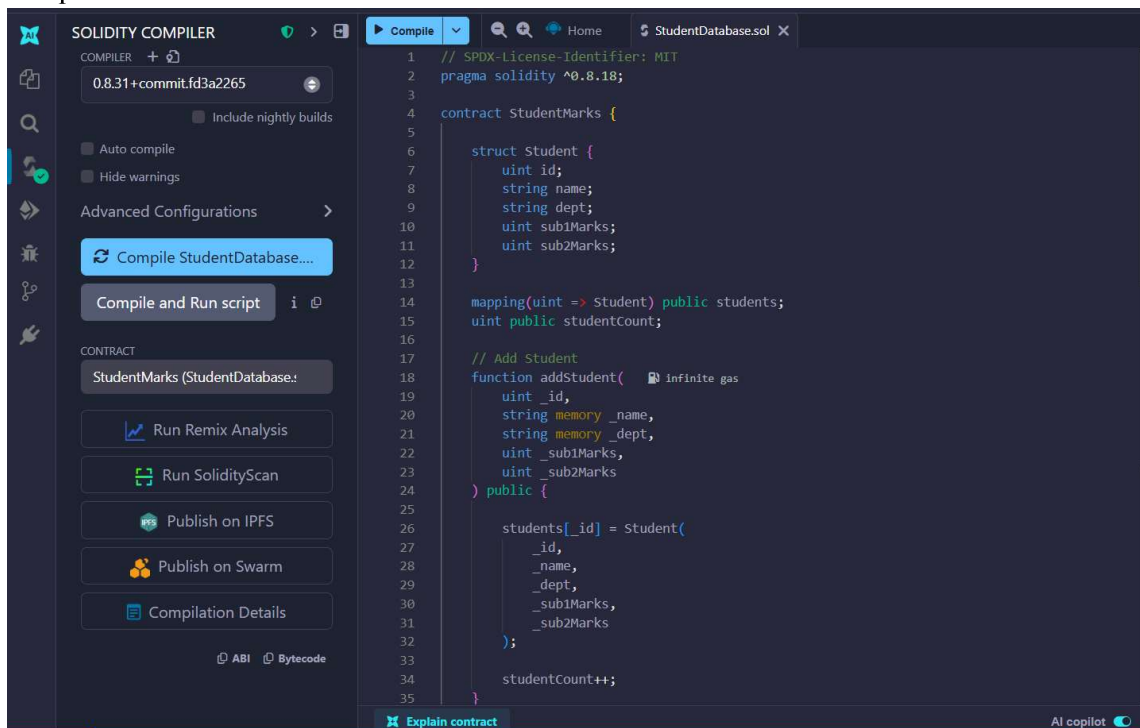
        students[_id] = Student(
            _id,
            _name,
            _dept,
            _sub1Marks,
            _sub2Marks
        );

        studentCount++;
    }
    // Get Student Details
    function getStudent(uint _id)
        public
        view
        returns (
            uint,
            string memory,
            string memory,
            uint,
            uint,
            uint
        )
    {
        Student memory s = students[_id];
        uint total = s.sub1Marks + s.sub2Marks;
        return (
            s.id,
            s.name,
            s.dept,
            s.sub1Marks,
```

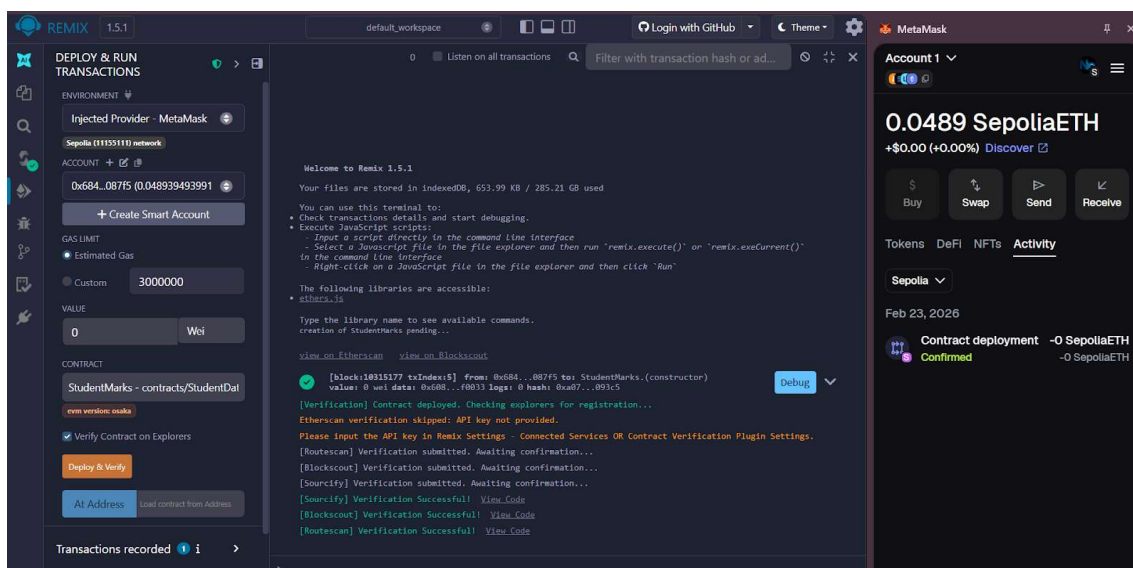
```
    s.sub2Marks,  
    total  
  );  
}  
}
```



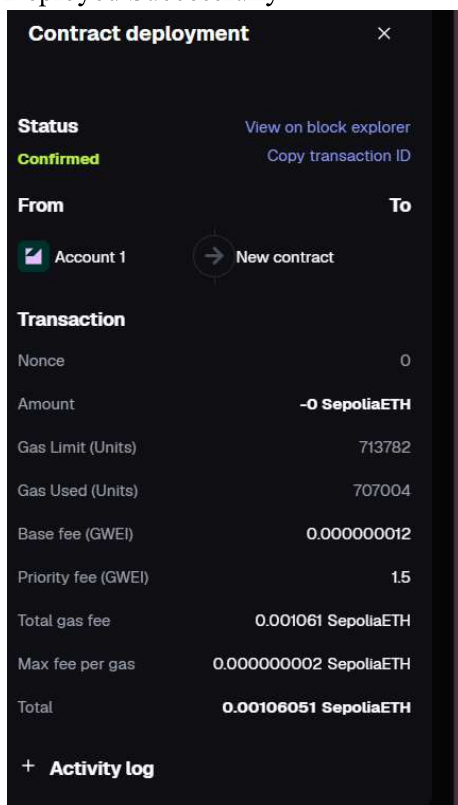
Recipient: 0x684Cae2603358E792862f69EC0B6d86cC79087f5



Compiled successfully



Deployed Successfully



Txn ID:

0xa66df353c71bc94d1426df4b9ffe4685f92cee72824b24156f1cd58e589fa148

Block Hash: 0xa07cc4fa138669edf381c141de461d0692f9814fad6a7e2013ae60c7664093c5

Contract Address:

0x06c523dc55770601613Fe5F6ab788ec1B44eAb55

From:

0x684Cae2603358E792862f69EC0B6d86cC79087f5

To:

0x684Cae2603358E792862f69EC0B6d86cC79087f5

✓

[block:10315177 txindex:5] from: 0x684...08/f5 to: StudentMarks.(constructor)
value: 0 wei data: 0x608...f0033 logs: 0 hash: 0xa07...093c5

Debug

status

1 Transaction mined and execution succeed

transaction hash

0xa66df353c71bc94d1426df4b9ffe4685f92cee72824b24156f1cd58e589fa148

block hash

0xa07cc4fa138669edf381c141de461d0692f9814fad6a7e2013ae60c7664093c5

block number

10315177

contract address

0x06c523dc55770601613fe5f6ab788ec1b44eab55

from

0x684Cae2603358E792862f69EC0B6d86cC79087f5

to

StudentMarks.(constructor)

transaction cost

707004 gas

decoded input

{}

decoded output

-

logs

[]

raw logs

[]

TRANSACTION ACTION

Call 0x60806040 Method by 0x684Cae26...cc79087f5

[This is a Sepolia Testnet transaction only]

Transaction Hash:

0xa66df353c71bc94d1426df4b9ffe4685f92cee72824b24156f1cd58e589fa148

Status:

Success

Block:

10315177 6 Block Confirmations

Timestamp:

1 min ago (Feb-22-2026 06:40:36 PM UTC)

From:

0x684Cae2603358E792862f69EC0B6d86cC79087f5

To:

[0x06c523dc55770601613fe5f6ab788ec1b44eab55 Created]

Value:

0 ETH

Transaction Fee:

0.001060506008484048 ETH

Gas Price:

1.500000012 Gwei (0.000000001500000012 ETH)

USING SMART CONTRACT:

Deployed Contracts 1

STUDENTMARKS AT 0X06C...EA

Balance: 0 ETH

ADDSTUDENT

_id: 221009

_name: tanmay

_dept: INFT

_sub1Marks: 100

_sub2Marks: 100

Call data Parameters

transact

getStudent uint256 _id

studentCount

students uint256


```
Decoded Input: {
    "uint256_id": "221009",
    "string_name": "tanmay",
    "string_dept": "INFT",
    "uint256_sub1Marks": "100",
    "uint256_sub2Marks": "100"
}
```

```
call to StudentMarks.studentCount

CALL [call] from: 0x684cae2603358e792862f69ec0b6d86cc79087f5 to: StudentMarks.studentCount()
data: 0xe58...1f9ee [Debug]

from 0x684Cae2603358E792862f69EC0b6d86cC79087f5

to StudentMarks.studentCount() 0x06c523dc55770601613Fe5F6ab788ec1844eAb55

input 0xe58...1f9ee

output 0000000000000000000000000000000001

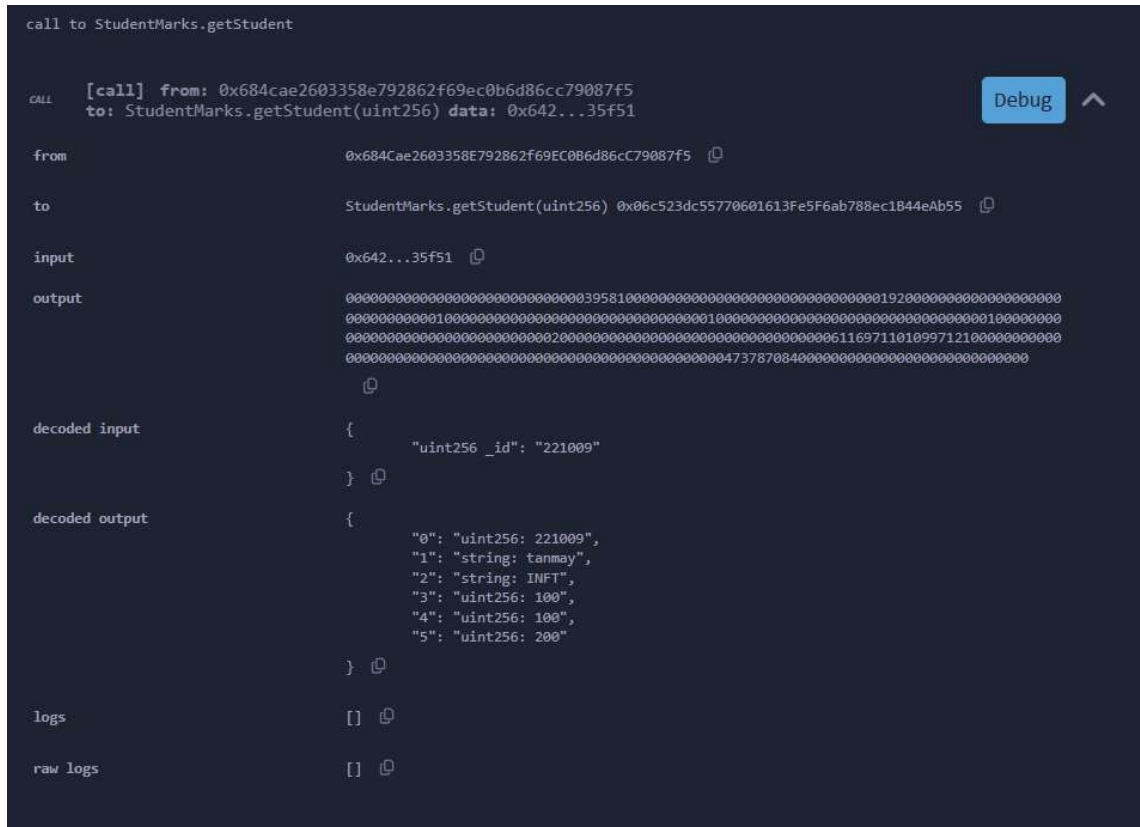
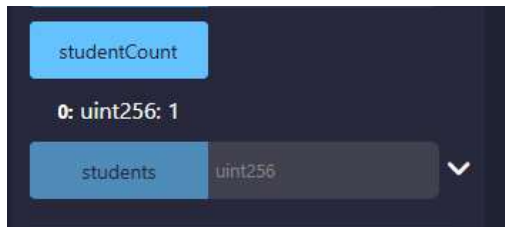
decoded input {}

decoded output {
  "0": "uint256: 1"
}

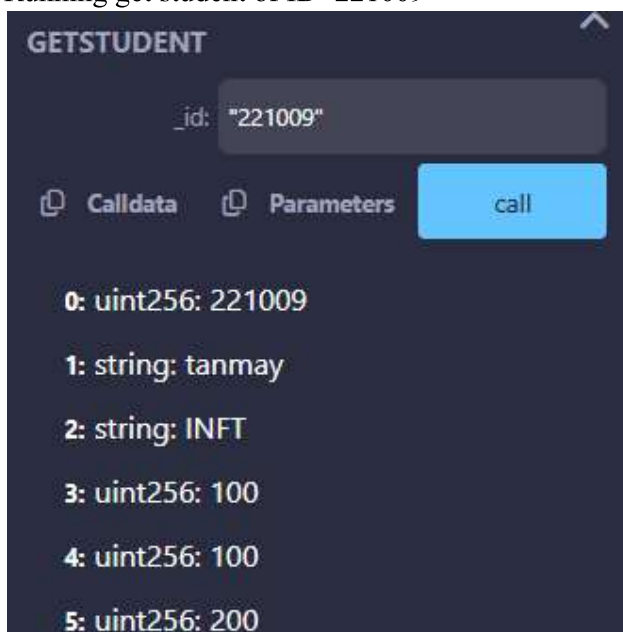
logs []

raw logs []
```

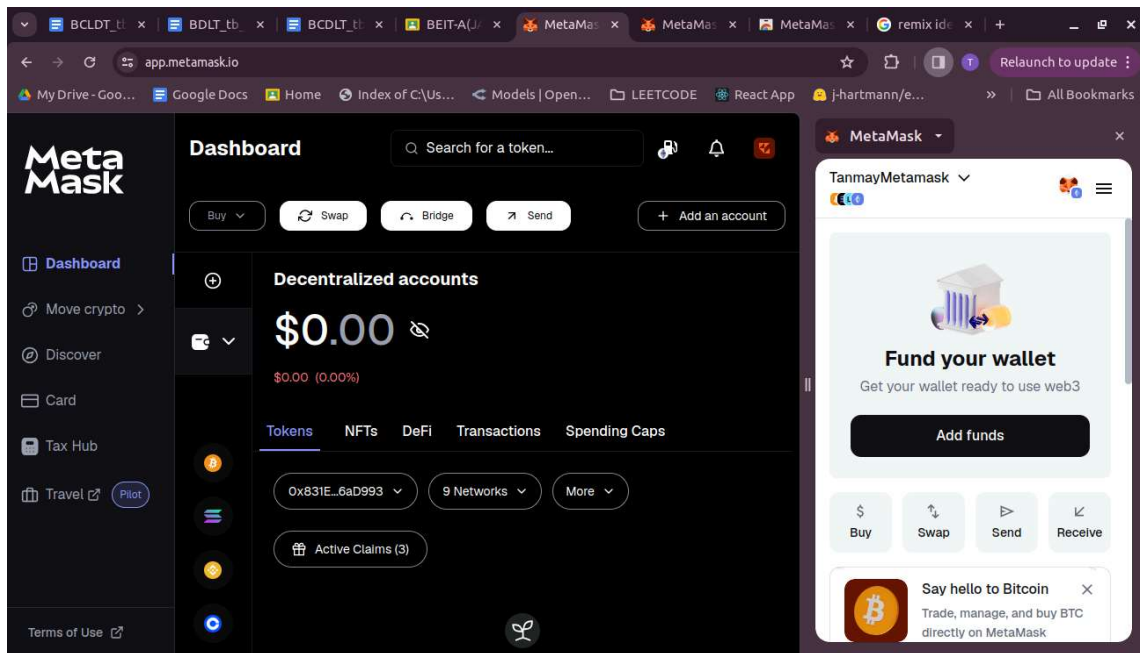
Finding Student Count



Running get student of ID=221009

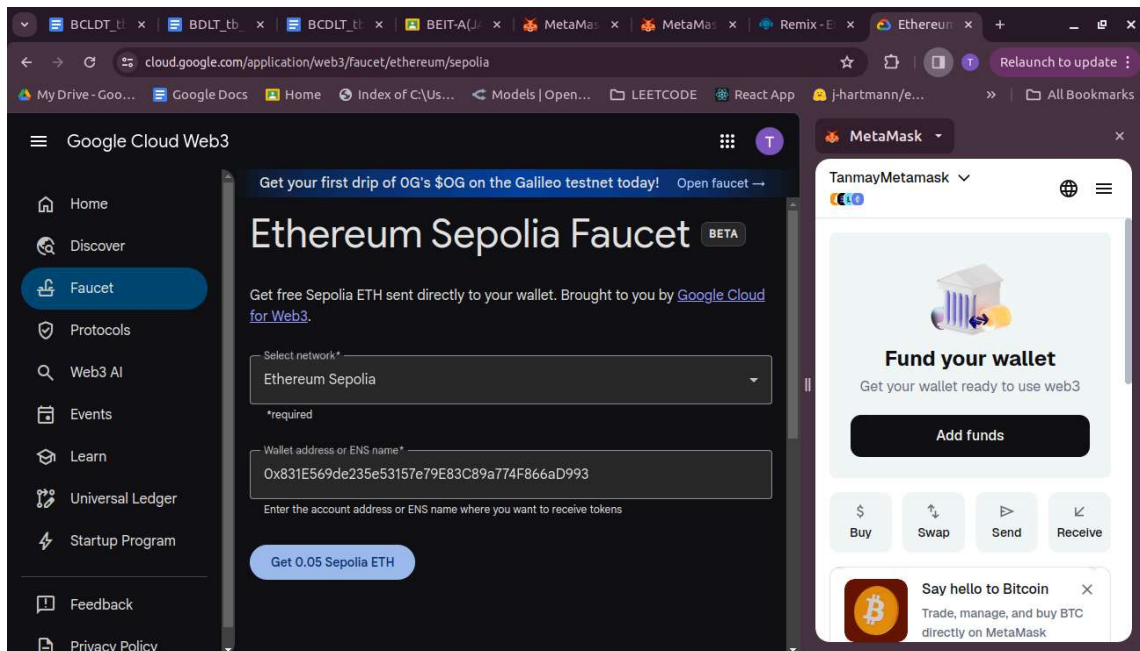


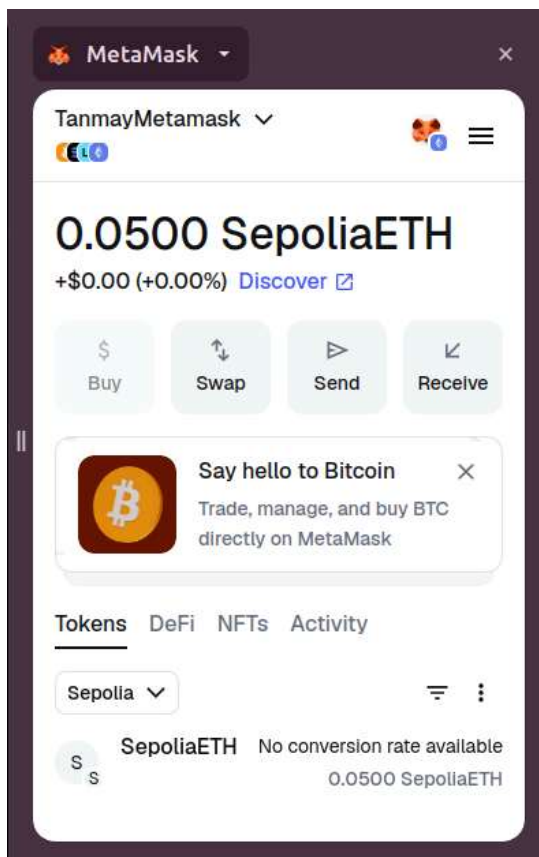
POST EXPERIMENT EXERCISE:



Metamask dashboard

Web3 Faucet





Create any one game using smart contract and deploy it.
MY PROGRAM:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.17;

contract RandomNumberGenerator {
    // Event to emit the random number
    event RandomNumberGenerated(uint256 randomNumber);

    // Function to generate a random number between 1 and the specified upper limit
    function generateRandomNumber(uint256 _upperLimit) public returns (uint256) {
        require(_upperLimit > 0, "Upper limit must be greater than 0");

        // Generate a pseudo-random number using block properties
        uint256 randomNumber = uint256(keccak256(abi.encodePacked(block.timestamp,
        block.difficulty))) % _upperLimit + 1;

        // Emit the random number
        emit RandomNumberGenerated(randomNumber);

        return randomNumber;
    }
}
```

The screenshot displays the Remix IDE interface, which is used for developing and deploying smart contracts on the Ethereum blockchain. The interface is divided into several panels:

- SOLIDITY COMPILER:** The left sidebar shows the compiler settings, including the Solidity version (0.8.31+commit.f3a2265) and options like "Auto compile" and "Include rightly builds".
- Contract Code:** The main editor displays the Solidity code for the `RandomNumberGenerator` contract. The code includes a pragma statement for Solidity 0.8.17, a constructor, and a function `generateRandomNumber` that uses a pseudo-random number generator.
- MetaMask:** The right sidebar shows the MetaMask wallet interface, displaying the balance of 0.0500 SepoliaETH and a transaction history.
- DEPLOY & RUN TRANSACTIONS:** The bottom panel shows the deployment process. The contract `RandomNumberGenerator - contract` is selected, and the estimated gas is 70000. The deployment is successful, and the transaction is confirmed on the Sepolia testnet.

The deployment log in the bottom panel shows the following details:

- Transaction:** [vm] from: 0x583...eddC4, to: RandomNumberGenerator, value: 0 wei, data: 0x608...f0833, logs: 0.
- Block:** [block:10173770 txIndex:21]
- Contract:** RandomNumberGenerator, value: 0 wei, data: 0x608...f0833, logs: 0.
- Hash:** 0x9c6...ee261.
- Verification:** Contract deployed, Checking explorers for registration...